Creating Firebase Collections

Command Line Tutorial

Learn to manage Firestore databases from the terminal

Learning Objectives

By the end of this session, you will be able to:

- ✅ **Install and configure** Firebase CLI
- ✅ **Create Firebase projects** from command line
- ✅ **Initialize Firestore** database
- ✅ **Create collections** and documents via CLI
- ✅ **Import/export data** using Firebase tools
- ✅ **Use Firebase emulators** for development

Why Learn Command Line Firebase?

🚀 Professional Development Benefits

- **Automation**: Script database operations
- **CI/CD Integration**: Deploy databases with code
- **Bulk Operations**: Import large datasets efficiently
- **Version Control**: Track database schema changes
- **Team Collaboration**: Consistent development environments

📚 Educational Value

- **Understanding Infrastructure**: How databases are really created
- **DevOps Skills**: Modern deployment practices
- **Scripting Knowledge**: Automation capabilities

Prerequisites

Required Tools

- ✅ **Node.js** (v16 or higher)
- ✅ **npm** or **yarn** package manager
- ✅ **Google Account** for Firebase
- ✅ **Terminal/Command Prompt** access

Recommended Knowledge

- Basic command line navigation
- Understanding of JSON structure
- Familiarity with Firebase concepts (from previous lessons)

Step 1: Install Firebase CLI

Installation Command

```
# Install Firebase CLI globally
npm install -g firebase-tools

# Verify installation
firebase --version
```

Expected Output

```
12.9.1
```

Alternative Installation Methods

```
# Using yarn
yarn global add firebase-tools
```

Step 2: Login to Firebase

Login Command

```
firebase login
```

What Happens

1. **Browser Opens**: Automatic redirect to Google login
2. **Permission Grant**: Allow Firebase CLI access
3. **Success Message**: "✔ Success! Logged in as your-email@gmail.com"

Verify Login

```
firebase projects:list
```

Expected Output

Step 3: Create a New Firebase Project

Using Firebase Console (Web)

```
# Open Firebase Console
firefox https://console.firebase.google.com
# or
open https://console.firebase.google.com
```

Create Project Steps

1. Click **"Create a project"**
2. Enter project name: `"foobar-database-demo"`
3. Choose **Google Analytics** settings (optional for learning)
4. Wait for project creation
5. Note your **Project ID**

Step 4: Initialize Local Firebase Project

Create Project Directory

```
# Create and navigate to project directory
mkdir firebase-cli-demo
cd firebase-cli-demo
```

Initialize Firebase

```
firebase init
```

Interactive Setup

```
? Which Firebase features do you want to set up?
❯○ Realtime Database
 ◉ Firestore
 ○ Functions
 ○ Hosting
```

Step 5: Firestore Initialization

Project Selection

```
? Please select an option:
❯ Use an existing project
  Create a new project
  Add Firebase to an existing Google Cloud Platform project
```

Choose Your Project

```
? Select a default Firebase project for this directory:
❯ foobar-database-demo (foobar-database-demo)
  [other projects...]
```

Firestore Rules

```
? What file should be used for Firestore Rules?
❯ firestore.rules
```

Step 6: Project Structure Created

Generated Files

```
firebase-cli-demo/
├── .firebaserc                # Project configuration
├── firebase.json              # Firebase settings
├── firestore.rules            # Security rules
└── firestore.indexes.json     # Database indexes
```

Verify Configuration

```
# Check current project
firebase use

# Output: Active Project: foobar-database-demo (foobar-database-demo)
```

Step 7: Understanding Firestore Rules

Default Rules (firestore.rules)

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    // Allow read/write access on all documents to any user signed in to the application
    match /{document=**} {
      allow read, write: if request.auth != null;
    }
  }
}
```

Open Rules for Development

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    // WARNING: These rules allow anyone on the internet to edit your database!
```

Step 8: Deploy Firestore Rules

Deploy Command

```
firebase deploy --only firestore:rules
```

Expected Output

```
=== Deploying to 'foobar-database-demo'...

i  deploying firestore
i  firestore: reading indexes from firestore.indexes.json...
✔  firestore: deployed indexes
i  firestore: reading rules from firestore.rules...
✔  firestore: released rules

✔  Deploy complete!

Project Console: https://console.firebase.google.com/project/foobar-database-demo/overview
```

Step 9: Start Firebase Emulators

Initialize Emulators

```
firebase init emulators
```

Select Emulators

```
? Which Firebase emulators do you want to set up?
❯◉ Authentication Emulator
 ◉ Functions Emulator
 ◉ Firestore Emulator
 ○ Database Emulator
 ○ Hosting Emulator
 ○ Pub/Sub Emulator
 ○ Storage Emulator
```

Start Emulators

Step 10: Create Collections via Emulator UI

Open Emulator UI

```
# Open in browser
open http://127.0.0.1:4000/firestore
```

Create Collection Manually

1. Click **"Start collection"**
2. Collection ID: `foobars`
3. Document ID: Leave auto-generated
4. Add fields:
    - `foo` (string): "Hello World"
    - `bar` (number): 42

Step 11: Create Data Files for Import

Create Sample Data (data.json)

```
{
  "foobars": {
    "doc1": {
      "foo": "Student",
      "bar": 2024,
      "createdAt": "2024-01-15T10:00:00Z"
    },
    "doc2": {
      "foo": "Teacher",
      "bar": 2023,
      "createdAt": "2024-01-15T10:01:00Z"
    },
    "doc3": {
      "foo": "Course",
      "bar": 456,
      "createdAt": "2024-01-15T10:02:00Z"
    }
  }
```

Step 12: Import Data Using Firebase CLI

Method 1: Direct Firebase Import

```
# Note: This requires specific JSON format
firebase firestore:delete --all-collections --force
firebase firestore:import data.json
```

Method 2: Using Node.js Script

```
// import-data.js
const admin = require('firebase-admin');
const serviceAccount = require('./serviceAccountKey.json');

admin.initializeApp({
  credential: admin.credential.cert(serviceAccount)
});

const db = admin.firestore();

async function importData() {
  const batch = db.batch();

  const data = [
    { foo: 'Student', bar: 2024 },
    { foo: 'Teacher', bar: 2023 },
    { foo: 'Course', bar: 456 }
```

Step 13: Run Import Script

Install Dependencies

```
npm init -y
npm install firebase-admin
```

Get Service Account Key

1. Go to **Firebase Console** → **Project Settings**
2. **Service Accounts** tab
3. **Generate new private key**
4. Save as `serviceAccountKey.json`

Run Import

```
node import-data.js
```

Step 14: Verify Collections Created

Using Firebase CLI

```
# List all collections (requires custom script)
firebase firestore:list
```

Using Emulator UI

- Open http://127.0.0.1:4000/firestore
- See `foobars` collection with 3 documents

Using Firebase Console

- Visit https://console.firebase.google.com
- Select your project
- Go to **Firestore Database**

Step 15: Command Line Queries

Create Query Script (query.js)

```
const admin = require('firebase-admin');
admin.initializeApp();

const db = admin.firestore();

async function queryCollection() {
  console.log('📖 Reading all documents from foobars collection:');

  const snapshot = await db.collection('foobars').get();

  snapshot.forEach(doc => {
    console.log(`📄 ${doc.id}:`, doc.data());
  });

  console.log(`\n📊 Total documents: ${snapshot.size}`);
}

async function queryWithFilter() {
  console.log('\n🔍 Querying documents where bar > 500:');

  const snapshot = await db.collection('foobars')
    .where('bar', '>', 500)
    .get();

  snapshot.forEach(doc => {
    console.log(`📄 ${doc.id}:`, doc.data());
  });
}

async function main() {
  await queryCollection();
  await queryWithFilter();
  process.exit(0);
```

Step 16: Advanced Collection Management

Create Multiple Collections Script

```
// create-collections.js
const admin = require('firebase-admin');
admin.initializeApp();

const db = admin.firestore();

async function createCollections() {
  const collections = [
    {
      name: 'users',
      data: [
        { name: 'John Doe', email: 'john@example.com', role: 'student' },
        { name: 'Jane Smith', email: 'jane@example.com', role: 'teacher' }
      ]
    },
    {
      name: 'courses',
      data: [
        { title: 'Database Design', code: 'ASE456', credits: 3 },
        { title: 'Software Engineering', code: 'ASE123', credits: 4 }
      ]
    }
  ];

  for (const collection of collections) {
    console.log(`📝 Creating collection: ${collection.name}`);

    const batch = db.batch();

    collection.data.forEach((item, index) => {
      const docRef = db.collection(collection.name).doc();
      batch.set(docRef, {
        ...item,
        createdAt: admin.firestore.FieldValue.serverTimestamp(),
        id: docRef.id
      });
    });

    await batch.commit();
    console.log(`✅ Created ${collection.data.length} documents in ${collection.name}`);
  }
```

Step 17: Collection Backup and Export

Export Data

```
# Export entire database
firebase firestore:export gs://your-bucket-name/backup-folder

# Export specific collection
firebase firestore:export gs://your-bucket-name/backup-folder --collection-ids=foobars
```

Local Export Script

```
// export-data.js
const admin = require('firebase-admin');
const fs = require('fs');

admin.initializeApp();
const db = admin.firestore();

async function exportCollection(collectionName) {
  console.log(`📤 Exporting collection: ${collectionName}`);

  const snapshot = await db.collection(collectionName).get();
```

Step 18: Delete Collections

Delete Collection Script

```
// delete-collection.js
const admin = require('firebase-admin');
admin.initializeApp();

const db = admin.firestore();

async function deleteCollection(collectionName) {
  console.log(`🗑️  Deleting collection: ${collectionName}`);

  const collectionRef = db.collection(collectionName);
  const batchSize = 500;

  const snapshot = await collectionRef.limit(batchSize).get();

  if (snapshot.size === 0) {
    console.log('✅ Collection is already empty');
    return;
  }

  const batch = db.batch();
  snapshot.docs.forEach(doc => {
    batch.delete(doc.ref);
  });

  await batch.commit();
  console.log(`✅ Deleted ${snapshot.size} documents`);

  // Recursively delete remaining documents
  if (snapshot.size === batchSize) {
    await deleteCollection(collectionName);
  }
}
```

Step 19: Useful Firebase CLI Commands

Project Management

```
# List projects
firebase projects:list

# Switch project
firebase use project-id

# Get current project
firebase use
```

Firestore Operations

```
# Deploy security rules
firebase deploy --only firestore:rules

# Deploy indexes
firebase deploy --only firestore:indexes
```

Step 20: Automation Scripts

Complete Setup Script (setup.sh)

```
#!/bin/bash

echo "🚀 Setting up Firebase project for development"

# Check if Firebase CLI is installed
if ! command -v firebase &> /dev/null; then
    echo "❌ Firebase CLI not found. Installing..."
    npm install -g firebase-tools
fi

# Initialize Firebase project
echo "📝 Initializing Firebase project..."
firebase init firestore emulators

# Create sample data
echo "📊 Creating sample data..."
cat > sample-data.json << EOF
{
  "foobars": {
    "doc1": {"foo": "Hello", "bar": 42},
    "doc2": {"foo": "World", "bar": 100},
    "doc3": {"foo": "Firebase", "bar": 2024}
  }
}
EOF

# Start emulators
echo "🔥 Starting Firebase emulators..."
firebase emulators:start --detach

echo "✅ Setup complete!"
```

Step 21: Best Practices

Security Rules Development

```
// Good: Restrictive rules for production
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Users can only read/write their own documents
    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }

    // Public read, authenticated write
    match /courses/{document} {
      allow read: if true;
      allow write: if request.auth != null;
    }
  }
}
```


Environment Management

Step 22: Integration with CI/CD

GitHub Actions Workflow

```yaml
# .github/workflows/firebase.yml
name: Deploy to Firebase

on:
  push:
    branches: [ main ]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v3

    - name: Setup Node.js
      uses: actions/setup-node@v3
      with:
        node-version: '18'

    - name: Install dependencies
      run: npm install

    - name: Install Firebase CLI
      run: npm install -g firebase-tools

    - name: Deploy to Firebase
      run: firebase deploy --token "$FIREBASE_TOKEN"
```

Step 23: Troubleshooting Common Issues

Authentication Problems

```
# Re-login to Firebase
firebase logout
firebase login

# Check authentication status
firebase login:list
```

Permission Errors

```
# Check project permissions
firebase projects:list

# Ensure you have correct project selected
firebase use --add
```

Emulator Issues

Step 24: Monitoring and Analytics

View Usage Statistics

```
# Check Firestore usage (requires billing account)
firebase projects:list
```

Monitor via Console

- Visit **Firebase Console**
- Select **Firestore Database**
- Check **Usage** tab for:
  - Document reads/writes
  - Storage usage
  - Bandwidth usage

Set Up Alerts

Practice Exercises

Beginner Level

1. **Setup Exercise**: Install Firebase CLI and create your first project
2. **Collection Creation**: Create a `students` collection with sample data
3. **Basic Queries**: Write scripts to read and filter data

Intermediate Level

1. **Data Migration**: Import data from CSV to Firestore
2. **Backup System**: Create automated backup scripts
3. **Multi-Environment**: Set up development/production environments

Advanced Level

1. **CI/CD Pipeline**: Set up automated deployment

Summary & Key Takeaways

✅ What You've Learned

- Firebase CLI installation and configuration
- Creating and managing Firestore collections via command line
- Importing/exporting data programmatically
- Using Firebase emulators for development
- Best practices for production deployments

🚀 Benefits of CLI Approach

- **Automation**: Scriptable database operations
- **Version Control**: Track database schema changes
- **Team Collaboration**: Consistent development environments
- **Professional Skills**: Industry-standard DevOps practices

Resources & References

Documentation

- Firebase CLI Reference
- Firestore Import/Export
- Firebase Emulator Suite

Tools

- Firebase Console
- Firebase CLI GitHub
- Firebase Admin SDK

Community

- Firebase Discord

Questions & Discussion

Reflection Questions

1. When would you use CLI tools vs GUI interfaces?
2. How does command line access improve development workflow?
3. What are the security implications of CLI access?

Practical Scenarios

1. How would you migrate data from SQLite to Firestore?
2. How would you set up a staging environment?
3. How would you automate daily backups?

Thank you for learning!

Ready to automate your Firebase workflows? 🔥⚡